



UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Big Data Processing

...

PySpark SQL

LAST WEEK RECAP

train_id	station	time
324	San Francisco	7:59a
324	22nd Street	8:03a
324	Millbrae	8:16a
324	Hillsdale	8:24a
324	Redwood City	8:31a
324	Palo Alto	8:37a
324	San Jose	9:05a
217	Gilroy	6:06a
217	San Martin	6:15a
217	Morgan Hill	6:21a
217	Blossom Hill	6:36a
217	Capitol	6:42a
217	Tamien	6:50a
217	San Jose	6:59a

train_id	station	time
324	San Francisco	7:59a
324	22nd Street	8:03a
324	Millbrae	8:16a
324	Hillsdale	8:24a
324	Redwood City	8:31a
324	Palo Alto	8:37a
324	San Jose	9:05a



train_id	station	time
217	Gilroy	6:06a
217	San Martin	6:15a
217	Morgan Hill	6:21a
217	Blossom Hill	6:36a
217	Capitol	6:42a
217	Tamien	6:50a
217	San Jose	6:59a

Creating Temp Tables

Creating temp tables

```
# Initialize Spark session
spark = SparkSession.builder.appName("Spark SQL Example").getOrCreate()

# Sample DataFrame
data = [("Alice", "HR", 30), ("Bob", "IT", 40), ("Cathy", "HR", 28)]
columns = ["Name", "Department", "Age"]
df = spark.createDataFrame(data, schema=columns)

# Register DataFrame as a temporary view
df.createOrReplaceTempView("people")

# Query using SQL
result = spark.sql("SELECT Name, Age FROM people WHERE Age > 30")
result.show()
```

Example 1: Querying on a temp view

Temporary views are powerful tools that allow you to query data using SQL syntax, making complex data manipulations easier and more intuitive.

You already have a SparkContext, spark, and a PySpark DataFrame, df, available in your workspace.

- Register a new view called "data_view" from the DataFrame df.
- Run the provided SQL query to calculate total salary by position.

```
1 # Register as a view
2 df.createOrReplaceTempView("data_view")
3
4 # Advanced SQL query: Calculate total salary by Position
5 result = spark.sql("""
6     SELECT Position, SUM(Salary) AS Total_Salary
7     FROM data_view
8     GROUP BY Position
9     ORDER BY Total_Salary DESC
10 """)
11 )
12 result.show()
```

Position	Total_Salary
Engineer	2544679
Coordinator	1752702
Director	1702050
Specialist	1479329
Manager	1475810
Analyst	1137642

Example 2: Running SQL Queries Programmatically

DataFrames can be easily manipulated using SQL queries in PySpark.

Create a temporary table of DataFrame that you have created previously, then construct a query to select the names of the people from the temporary table and assign the result to a new DataFrame.

- Create a temporary table people.
- Construct a query to select the names of the people from the temporary table people.
- Assign the result of Spark's query to a new DataFrame - people_df_names.
- Print the top 10 names of the people from people_df_names DataFrame.

```
1 # Create a temporary table "people"
2 people_df.createOrReplaceTempView("people")
3
4 # Construct a query to select the names of the people from the temporary table "people"
5 query = '''SELECT name FROM people'''
6
7 # Assign the result of Spark's query to people_df_names
8 people_df_names = spark.sql(query)
9
10 # Print the top 10 names of the people
11 people_df_names.show(10)
```

Example 3: SQL queries for filtering Table

In the previous exercise, you have run a simple SQL query on a DataFrame. There are more sophisticated queries you can construct to obtain the result that you want and use it for downstream analysis such as data visualization and Machine Learning. In this exercise, we will use the temporary table people that you have created previously, filter out the rows where the "sex" is male and female and create two DataFrames.

Please note the "solution" is case sensitive for the SQL commands (For example, it only accepts FROM and not from). The "solution" only accepts "==" and not "=".

Remember, you already have a SparkSession spark and a temporary table people available in your workspace.

- Filter the people table to select all rows where sex is female into people_female_df DataFrame.
- Filter the people table to select all rows where sex is male into people_male_df DataFrame.
- Count the number of rows in both people_female and people_male DataFrames.

```
1 # Filter the people table to select female sex
2 people_female_df = spark.sql('SELECT * FROM people WHERE sex=="female"')
3
4 # Filter the people table DataFrame to select male sex
5 people_male_df = spark.sql('SELECT * FROM people WHERE sex=="male"')
6
7 # Count the number of rows in both people_df_female and people_male_df DataFrames
8 print("There are {} rows in the people_female_df and {} rows in the people_male_df DataFrames".format
    (people_female_df.count(), people_male_df.count()))
```

There are 49014 rows in the people_female_df and 49066 rows in the people_male_df DataFrames

Example 4: Analytics with SQL on DataFrames

SQL queries are concise and easy to run compared to DataFrame operations. But in order to apply SQL queries on a DataFrame first, you need to create a temporary view of the DataFrame as a table and then apply SQL queries on the created table.

You already have a SparkContext spark and salaries_df available in your workspace.

- Create temporary table "salaries_table" from salaries_df DataFrame.
- Construct a query to extract the "job_title" column from company_location in Canada ("CA").
- Apply the SQL query and create a new DataFrame canada_titles.
- Get a summary of the table.

```
1 # Create a temporary view of salaries_table
2 salaries_df.createOrReplaceTempView('salaries_table')
3
4 # Construct the "query"
5 query = '''SELECT job_title, salary_in_usd FROM salaries_table WHERE company_location == "CA"'''
6
7 # Apply the SQL "query"
8 canada_titles = spark.sql(query)
9
10 # Generate basic statistics
11 canada_titles.describe().show()
```

summary	job_title	salary_in_usd
count	1115	1115
mean	null	143228.7273542601
stddev	null	61839.75996920976
min	AI Architect	15000
max	Statistical Progr...	800000

Example 5: PySpark DataFrame visualization

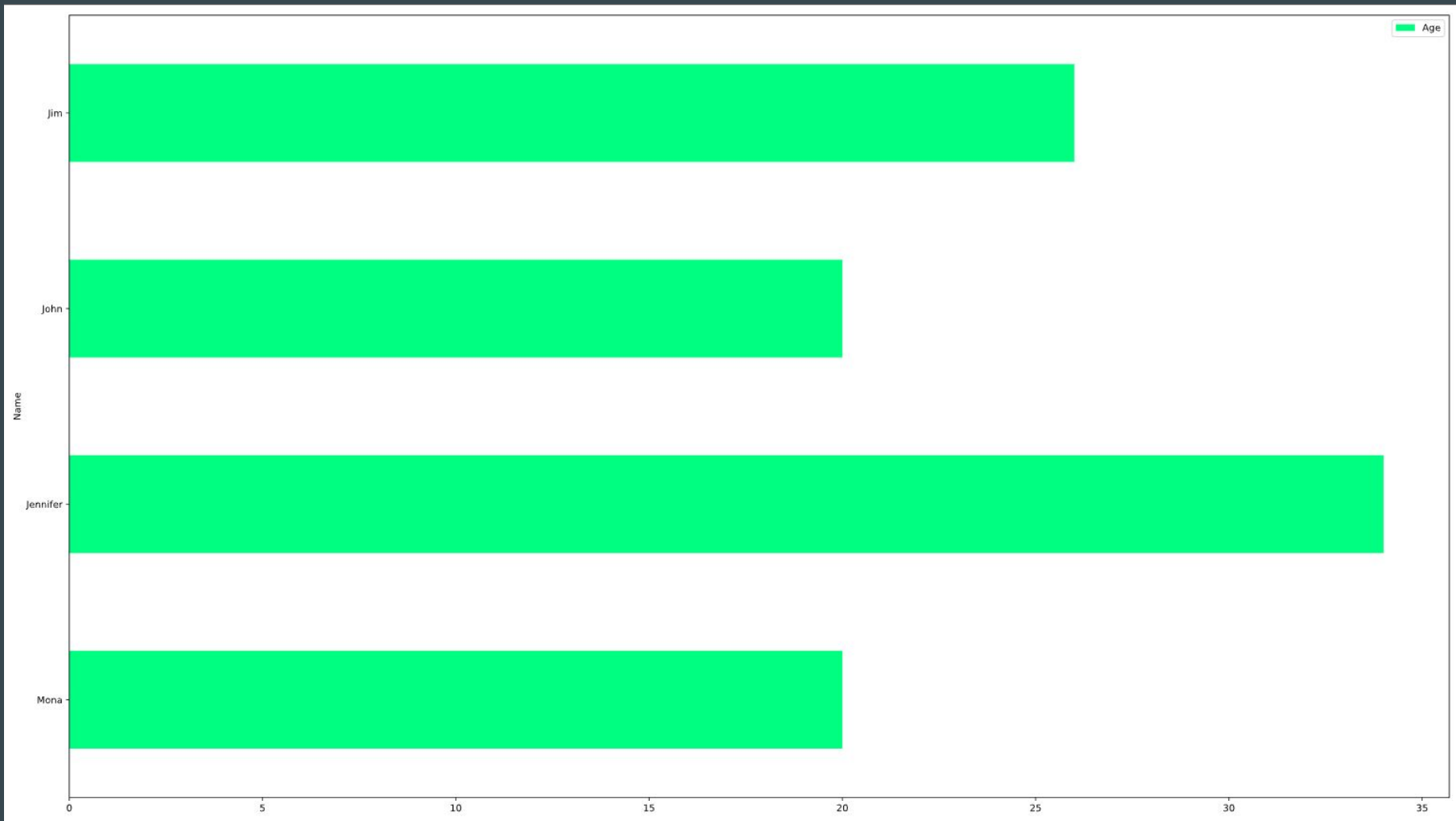
Graphical representations or visualization of data is imperative for understanding as well as interpreting the data. In this simple data visualization exercise, you'll first print the column names of `names_df` DataFrame that you have created earlier, then convert the `names_df` to Pandas DataFrame, and finally plot the contents as horizontal bar plot with names of the people on the x-axis and their age on the y-axis.

Remember, you already have a `SparkSession spark` and a `DataFrame names_df` available in your workspace.

- Print the names of the columns in `names_df` DataFrame.
- Convert `names_df` DataFrame to `df_pandas` Pandas DataFrame.
- Use matplotlib's `plot()` method to create a horizontal bar plot with 'Name' on x-axis and 'Age' on y-axis.

```
1 # Check the column names of names_df
2 print("The column names of names_df are",
   names_df.columns)
3
4 # Convert to Pandas DataFrame
5 df_pandas = names_df.toPandas()
6
7 # Create a horizontal bar plot
8 df_pandas.plot(kind='barh', x='Name', y='Age',
   colormap='winter_r')
9 plt.show()
```

```
The column names of names_df are ['Name', 'Age']
```

Combining DataFrame and SQL operations

```
# Filter salaries over 3000
```

```
filtered_df = df.filter(df.Salary > 3000)
```

```
# Register filtered DataFrame as a view
```

```
filtered_df.createOrReplaceTempView("filtered_employees")
```

```
# Aggregate using SQL on the filtered view
```

```
spark.sql("""  
    SELECT Department, COUNT(*) AS Employee_Count  
    FROM filtered_employees  
    GROUP BY Department  
""").show()
```

Handling data types in aggregations

Example of type casting

```
data = [("HR", "3000"), ("IT", "4000"), ("Finance", "3500")]  
columns = ["Department", "Salary"]  
df = spark.createDataFrame(data, schema=columns)
```

Convert Salary column to integer

```
df = df.withColumn("Salary", df["Salary"].cast("int"))
```

Perform aggregation

```
df.groupBy("Department").sum("Salary").show()
```

RDDs for aggregations

```
# Example of aggregation with RDDs
```

```
rdd = df.rdd.map(lambda row: (row["Department"], row["Salary"]))
```

```
rdd_aggregated = rdd.reduceByKey(lambda x, y: x + y)
```

```
print(rdd_aggregated.collect())
```

Best practices for PySpark aggregations

- Filter early: Reduce data size before performing aggregations
- Handle data types: Ensure data is clean and correctly typed
- Avoid operations that use the entire dataset: Minimize operations like `groupBy()`
- Choose the right interface: Prefer DataFrames for most tasks due to their optimizations
- Monitor performance: Use `explain()` to inspect the execution plan and optimize accordingly

Key takeaways

- PySpark SQL Aggregations: Functions like `SUM()` and `AVERAGE()` for summarizing data
- DataFrames and SQL: Combining both approaches for flexible data manipulation
- Handling Data Types: Addressing issues with type mismatches during aggregations
- RDDs vs DataFrames: Understanding the trade-offs and choosing the right tool

Example 6: Aggregating in PySpark

A SparkSession called **spark** is already in your workspace, along with the Spark DataFrame **salaries_df**

- Find the minimum salary at a US, Small company - performing the filtering by referencing the column directly ("salary_in_usd"), not passing a SQL string.
- Find the maximum salary at a US, Large company, denoted by a "L" - performing the filtering by referencing the column directly ("salary_in_usd"), not passing a SQL string.

```
1 # Find the minimum salaries for small companies
2 salaries_df.filter(salaries_df.company_size == "S").groupBy().min("salary_in_usd").show()
3
4 # Find the maximum salaries for large companies
5 salaries_df.filter(salaries_df.company_size == "L").groupBy().max("salary_in_usd").show()
```

```
+-----+
|min(salary_in_usd)|
+-----+
|                15809|
+-----+
+-----+
|max(salary_in_usd)|
+-----+
|                423000|
+-----+
```


Example 7: Aggregating in RDDs

A Spark session called **spark** has already been made for you.

- Create an RDD from the provided DataFrame.
- Change Salary type to “int”
- Apply the provided Lambda Function to the keys of the RDD.
- Collect the results of the aggregation.

```
data = [("HR", "3000"), ("IT", "4000"), ("HR", "3500")]
columns = ["Department", "Salary"]
df = spark.createDataFrame(data, schema=columns)
```

```
6 df = df.withColumn("Salary", df["Salary"].cast("int"))
7
8 # Map the DataFrame to an RDD
9 rdd = df.rdd.map(lambda row: (row["Department"], row["Salary"]))
10
11 # Apply a lambda function to get the sum of the DataFrame
12 rdd_aggregated = rdd.reduceByKey(lambda x, y: x + y)
13
14 # Show the collected Results
15 print(rdd_aggregated.collect())
```

```
[('HR', 6500), ('IT', 4000)]
```

Example 8: Complex Aggregations (FYI)

- Calculate the average salaries of large US companies using the "salary_in_usd" column.
- Calculate the total salaries of large US companies.

```
1 # Average salaries at large us companies
2 large_companies=salaries_df.filter(salaries_df.company_size == "L").filter(salaries_df.company_location ==
  "US").groupBy().avg("salary_in_usd").show()
3
4 #set a large companies variable for other analytics
5 large_companies=salaries_df.filter(salaries_df.company_size == "L").filter(salaries_df.company_location ==
  "US")
6
7 # Total salaries in usd
8 large_companies.groupBy().sum("salary_in_usd").show()
```

	+-----+
	avg(salary_in_usd)
	+-----+
	165465.82197614992
	+-----+
	+-----+
	sum(salary_in_usd)
	+-----+
	194256875
	+-----+



UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Stay Curious. Questions?

Thank you for your attention.
Lorent Sinani



Scan to connect on LinkedIn